# Problem N

Um carregamento de Nlogs, principal produto de exportação de Nlogônia, está no porto, em contêineres, pronto para ser embarcado. Todos os contêineres têm as mesmas dimensões e são cubos. Os contêineres estão organizados no pátio do porto em L linhas e C colunas, num total de LC contêineres. Cada contêiner está marcado com um número de identificação distinto, de 1 a LC. Cada uma das L linhas de contêineres será embarcada em um navio distinto. Para facilitar o desembarque nos diversos países em que serão entregues, os contêineres de uma linha devem estar organizados de forma que os números de identificação estejam ordenados. Mais precisamente, a linha 1 foi organizada no pátio de forma a conter os contêineres identificados de 1 a C ordenados crescentemente, a linha 2 de forma a conter os contêineres de C + 1 a 2C (ordenados crescentemente), e assim por diante, até a linha L, organizada de forma a conter os contêineres de (L – 1)C + 1 a LC (ordenados crescentemente).

O guindaste de embarque é capaz de movimentar ou uma linha completa ou uma coluna completa de contêineres, não sendo capaz de movimentar outros tipos de agrupamentos ou contêineres individuais. Na noite anterior ao embarque, um grupo de estivadores operou os guindastes para trocar linhas e colunas do carregamento, como forma de protestar quanto aos baixos salários. A figura (b) acima mostra a configuração dos contêineres após a troca das linhas 1 e 4; a figura (c) mostra a configuração após mais uma troca, entre as colunas 2 e 3. O carregamento precisa ser embarcado ainda hoje, mas antes disso é necessário que os contêineres sejam reorganizados da forma descrita. Você deve escrever um programa que, dada a informação sobre a posição de cada contêiner após o protesto, determine se é possível recolocar os contêineres na forma originalmente prevista utilizando apenas os guindastes, e nesse caso calcular o menor número de trocas de linhas e colunas necessário para esse fim.

## Inputs

A primeira linha de um programa contém dois inteiros L ($1 \le L \le 300$) e C ($1 \le C \le 300$) indicando respectivamente o número de linhas e o número de colunas do carregamento. As L linhas seguintes descrevem a posição dos contêineres depois do protesto dos estivadores. Cada uma dessas L linhas contém C números inteiros $X_{l,c}$ ($1 \le X_{l,c} \le LC$) indicando a posição de um contêiner. Cada número inteiro entre 1 e LC aparece na entrada, em alguma das L linhas. É garantido que cada número na configuração apareça uma única vez cada e que todos os números entre 1 e LC aparecerão na mesma.

## Outputs

Seu programa deve produzir uma única linha, contendo um único inteiro, o número mínimo de trocas de linhas e colunas que devem ser realizadas pelo guindaste para recolocar os contêineres na posição original. Se não for possível colocar os contêineres na posição original, utilizando apenas trocas de linhas e colunas, imprima o caractere "*".

## Examples

| Exemplo de entrada 1 | Exemplo de saída 1 |
|---|---|
| 2 2 | 1 |
| 3 4 | |

| Exemplo de entrada 1 | Exemplo de saída 1 |
|---|---|
| 1 2 | |

| Exemplo de entrada 2 | Exemplo de saída 2 |
|---|---|
| 3 3 | * |
| 9 2 4 | |
| 5 8 7 | |
| 6 1 3 | |

| Exemplo de entrada 3 | Exemplo de saída 3 |
|---|---|
| 5 4 | 2 |
| 13 15 14 16 | |
| 5 7 6 8 | |
| 9 11 10 12 | |
| 1 3 2 4 | |
| 17 19 18 20 | |

## Code

```python
# Estrutura de dados utilizada: listas para representar permutações de linhas e colunas
# Tipo de algoritmo: análise de permutação e contagem de ciclos (perm cycles)

def count_cycles(perm):
    visited = [False] * len(perm)
    cycles = 0
    for i in range(len(perm)):
        if not visited[i]:
            cycles += 1
            j = i
            while not visited[j]:
                visited[j] = True
                j = perm[j]
    return len(perm) - cycles

L, C = map(int, input().split())
mat = [list(map(int, input().split())) for _ in range(L)]

# Calcula permutação de linhas
row_perm = []
for i in range(L):
    # O primeiro elemento da linha i deveria ser (i * C + 1)
    val = mat[i][0]
    correct_row = (val - 1) // C
    row_perm.append(correct_row)

# Calcula permutação de colunas
col_perm = []
for j in range(C):
    # O elemento da primeira linha, coluna j, deveria ser (j + 1)
    val = mat[0][j]
    correct_col = (val - 1) % C
    col_perm.append(correct_col)

# Verifica se a matriz pode ser obtida apenas trocando linhas e colunas
possible = True
for i in range(L):
    for j in range(C):
        expected = row_perm[i] * C + col_perm[j] + 1
        if mat[i][j] != expected:
            possible = False
            break
    if not possible:
        break

if not possible:
    print("*")
else:
    print(count_cycles(row_perm) + count_cycles(col_perm))
```